Proximity risk after go-arounds and touch-and-goes — terminal commands

Every command and option of run_proximity.py with examples. Run everything from the repository root with the virtual environment active. Method: FRAMEWORK.md.

1. Set-up (once per machine)

`git clone \`
`  https://github.com/sponbob3/CollisionRiskAlgo.git`
`cd CollisionRiskAlgo`
	Get the repository.

`python3.12 --version`	Use Python 3.12 (3.11 also works). If the command is not found, install Python 3.12 from python.org (or brew install python@3.12) and open a new terminal. Other versions can stay installed: macOS's own python3 may be too old (3.9), and Python 3.13+ is too new for some of the traffic library's dependencies.

`python3.12 -m venv .venv`
`source .venv/bin/activate`
`# Windows: .venv\Scripts\activate`
`python -m pip install --upgrade pip`
`python -m pip install -r requirements.txt`
	Create the virtual environment and install every dependency (the same packages as the go-around pipeline plus scipy, statsmodels, pyproj, airportsdata and pytest). 'python -m pip' works even where a bare 'pip' command is not found.

`source .venv/bin/activate`	Run this in every NEW terminal window, inside the repository, before any other command; the prompt then starts with (.venv).
`git pull`	Update to the latest version later (run inside the repository).

2. Data

`mkdir -p datasets/KMCO_2025Q1`
`cp -R /path/to/kmco_data/* \`
`  datasets/KMCO_2025Q1/`
	One file per day (.parquet preferred, .csv accepted) of OpenSky-style state vectors for ALL traffic around the airport, either directly in the folder or in subfolders (e.g. one per month). Columns: timestamp, icao24, callsign, latitude, longitude, altitude, geoaltitude, vertical_rate, groundspeed, track, onground; a position-time column (last_position) is used when present to drop rows that repeat an old position. The folder name is <ICAO>_<label>; the prefix selects airports/<ICAO>.yaml. datasets/ is gitignored.

`python run_proximity.py check-data KMCO_2025Q1`
`python run_proximity.py check-data \`
`  KMCO_2025Q1 --limit 5`
	Data audit only: days found, columns, per-day message and aircraft counts, gaps and receiver outages, altitude references, and whether the volume meets the baseline minimum (28 days, 90 recommended). Writes output/KMCO/proximity_risk/check_data_KMCO_2025Q1.csv.

3. Airport profile (automatic)

`python run_proximity.py KMCO_2025Q1`	Nothing to do by hand: when airports/KMCO.yaml does not exist, the first run (or check-data) creates it - runways and elevation from the OurAirports database (downloaded once, needs internet), timezone from the coordinates, preset and assume_arrivals_dataset detected from the data, the ADS-B coverage floor near the field measured from the data, and the airspace block (field elevation + 4,000 ft, checked against the data every run). Each detected value is printed and written into the file with its evidence.
`python run_proximity.py new-airport KMCO \` `  KMCO_2025Q1`	Create the profile without running the analysis (the dataset name is optional; without it the preset comes from the airport type).
`open airports/KMCO.yaml`	The one place to change any airport setting (timezone, preset, terrain, assume_arrivals_dataset, ceiling, flows, go-around overrides). It is never overwritten; delete it to regenerate.

4. Running the analysis

`python run_proximity.py`	Interactive: asks which dataset, shows what it found (days, airport profile, pairwise cache, go-around runs, stored baseline), asks what to produce (figures, per-event pictures, sensitivity, probability add-on), prints the equivalent command and starts.
`Run Proximity Analysis.command  \` `  (double-click in Finder)`	macOS: the same questions without opening Terminal yourself. It uses .venv when present. If macOS blocks it the first time: right-click > Open > Open. If it is not executable after a download: chmod +x "Run Proximity Analysis.command".
`python run_proximity.py KMCO_2025Q1`	The complete run: go-around detection -> output/KMCO/goaround/run_NN/, then baseline (built or reused), encounters, window metrics, every event, sensitivity sweep, tables, figures, per-event plots and summary.pdf -> output/KMCO/proximity_risk/run_NN/ (same NN).
`python run_proximity.py KMCO_2025Q1 --limit \` `  5 --no-sensitivity --no-plots`	Quick test on the first 5 days without the sweep or figures.
`python run_proximity.py KMCO_2025Q1 \` `  --workers 4`	Parallel day processing (default: CPU count - 1).
`python run_proximity.py KMCO_2025Q1 \` `  --goaround-report --calibrate`	Also write the go-around stage's figures, narrative report and PDF, and its plateau-duration calibration histogram.

`python run_proximity.py KMCO_2025Q1 --window 5`
`python run_proximity.py KMCO_2025Q1 \`
`  --window 15`
	Pre/post window length in minutes (default 10; the sweep runs 5 and 15 anyway).

`python run_proximity.py KMCO_2025Q1 \` `  --goaround-run 1 --no-sensitivity`	Re-run the analysis on existing go-around results and caches (fastest way to get a new summary.pdf).

`python run_proximity.py KMCO_2025Q1 \` `  --probability`	Optional add-on: also run the conflict-probability model; its outputs go to proximity_risk/run_NN/probability/ and are not part of the summary.
`python run_proximity.py KMCO_2025Q1 \` `  --include-ambiguous`	Events are go-arounds and touch-and-goes (each keeps its own name; results are also given per type). This adds ga_ambiguous go-arounds.
`python run_proximity.py KMCO_2025Q1 \` `  --no-event-plots`	Skip the one-PNG-per-event plots only (figures and summary.pdf are still produced).

5. Reusing runs

`python run_proximity.py KMCO_2025Q1 \` `  --goaround-run 3`	Reuse output/KMCO/goaround/run_03 instead of re-running go-around detection (the proximity run still gets a new NN).
`ls output/KMCO/proximity_risk/cache/2025Q1/`	Per-day pairwise caches (parameter-hashed). They are reused automatically by every run and sweep; delete the folder to force recomputation.

6. Baselines

`python run_proximity.py KMCO_2025Q1 \` `  --baseline-only`	Build (or reuse) and validate the baseline, write baseline_report.pdf, and stop. Stored in output/KMCO/proximity_risk/baseline/2025Q1/ (baseline.json, baseline_windows.parquet, error_model.csv, phase1_removed.csv, validation/).
`python run_proximity.py KMCO_2025Q1 \` `  --rebuild-baseline`	Force a rebuild even if the stored baseline matches the data and parameters (it is rebuilt automatically when either changes).
`python run_proximity.py KMCO_2025Q2 \` `  --baseline-from KMCO_2025Q1`	Score another dataset's events against the stored KMCO_2025Q1 baseline (same airport).

7. Checks and tests

`python -m pytest`	The whole test suite (analytic intervals vs brute force, missed passes, scenarios, pruning, probability vs Monte Carlo, baseline calibration, injected effect, go-around parity, traffic cross-check). About two minutes.
`python -m pytest -q -m 'not slow'`	Skip the end-to-end synthetic runs.
`python -m proximity_pipeline.validate_cpa \` `  KMCO_2025Q1 20250108 20250115`	Cross-check closest-approach distances against the traffic library on the named days.

`python tools/make_synthetic_dataset.py \`
`  KBNA_synth --days 6`
`python run_proximity.py KBNA_synth --limit 3`
	Synthetic traffic for a smoke test without real data (--intruder-rate and --post-ga-irr inject a known effect).

`python tools/make_terminal_commands_pdf.py`	Regenerate this PDF after changing the CLI.

8. Where things land

`output/KMCO/goaround/run_NN/`	Go-around pipeline outputs (all_approaches.csv, go_around_events.csv, summary.pdf, plots/, run_config.txt).
`output/KMCO/proximity_risk/run_NN/`	run_config.txt, ceiling_check.csv, data_quality.csv, window_metrics.parquet, encounters.parquet/.csv, go_around_risk.csv, change_summary.csv (main result: go-arounds vs normal landings), landing_comparison/ (before-window checks, pre-trend, placebo, regression, robustness, minute by minute), landing_windows.csv, go_around_aircraft.csv, results_primary.csv, equilibrium.csv, epoch.csv, sensitivity.csv, figures/, events/, summary.pdf (start here: key results and a glossary on page 1).

Options summary: --goaround-run N, --rebuild-baseline, --baseline-only, --baseline-from ICAO_label, --window MIN, --include-ambiguous, --no-sensitivity, --no-plots, --no-event-plots, --goaround-report, --calibrate, --limit N, --workers N. Sub-commands: new-airport ICAO, check-data ICAO_label [--limit N].